

Introduction to Parallel Computing

a.y. 2026 – 2027

Prof. Flavio Vella

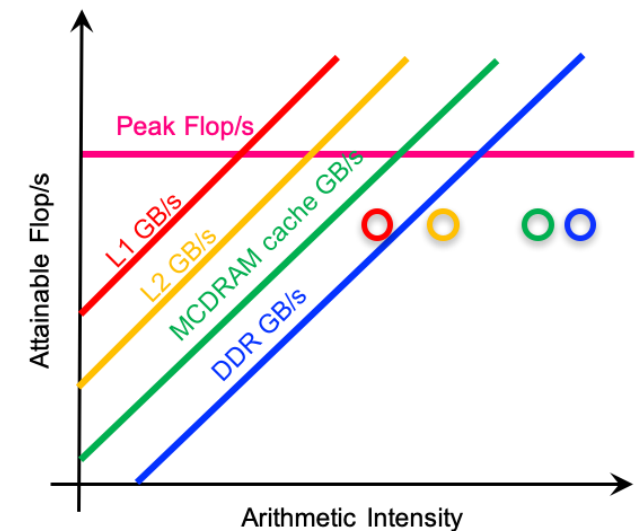
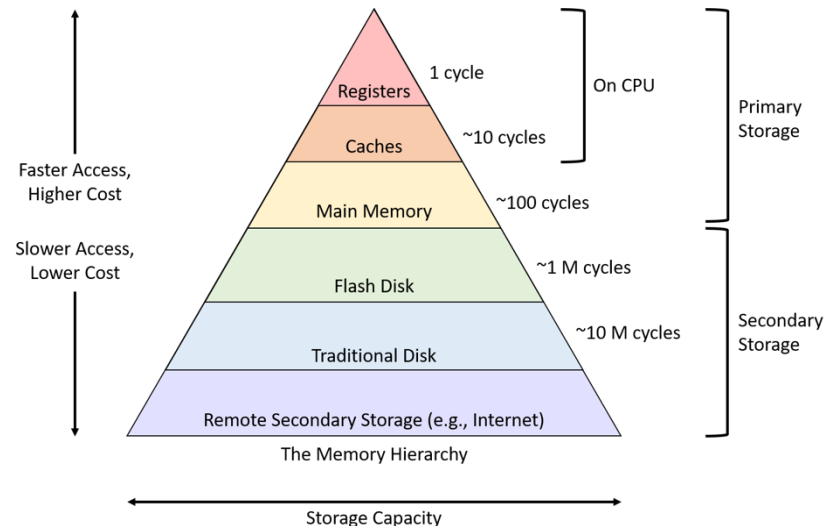
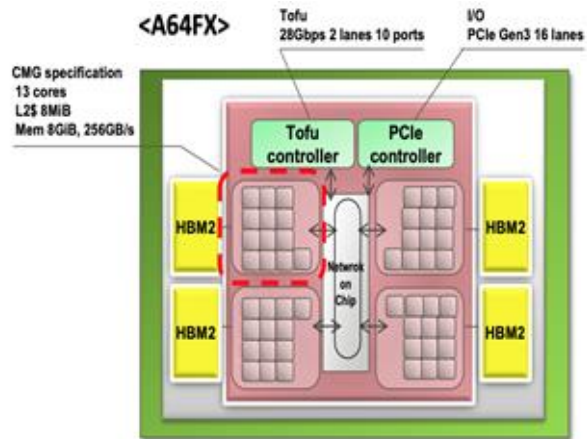
Lecture 3

SIMD and MIMD Architectures

From one datapath to parallel architectures

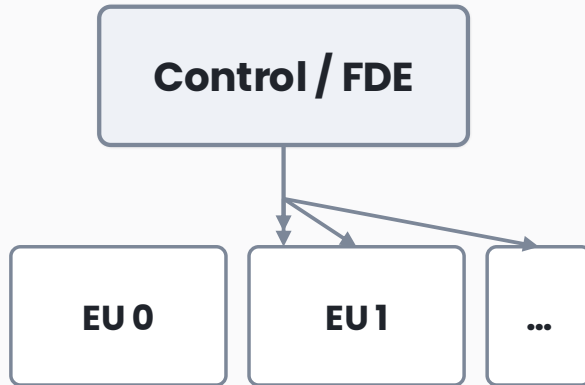
Performance in VN architectures depends on many factors (**all together**)

- FDE design and ISA (latency)
- ALU technology and PEs organization (math bandwidth)
- Memory sub-system (latency and memory bandwidth)
- Applications (ratio between math operations and read/write operations)



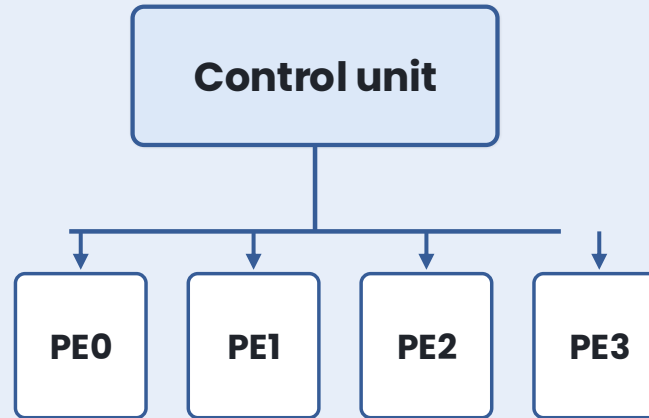
From one datapath to parallel architectures

L2: one instruction stream



More execution units can exploit ILP, but the program still provides one logical instruction stream.

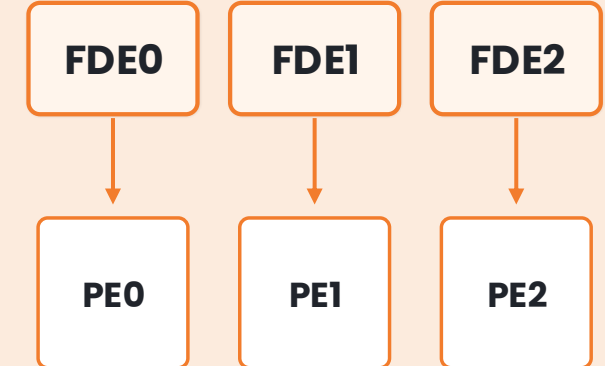
One control, many datapaths



One instruction is broadcast. Active datapaths apply it to different data elements.

SIMD

Independent control + datapath



Each processing unit can progress through its own instruction stream.

MIMD

Key idea: parallelism depends not only on how many execution units exist, but also on how instruction streams control them.

SIMD Architecture

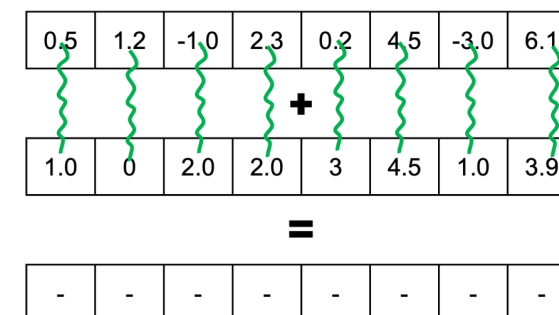
SIMD architectures are computer processor designs that allow a single instruction to be executed on multiple pieces of data simultaneously.

- They are designed to exploit **data-level parallelism**

Concurrency arises from performing the same operation on different pieces of data

Typical applications are

- Element-wise operations (e.g., certain types of encryption.)
- Matrix-centric scientific computing
- Media-oriented image and sound processors



SIMD Architecture

SIMD architectures are computer processor designs that allow a single instruction to be executed on multiple pieces of data simultaneously.

- They are designed to exploit **data-level parallelism**

SIMD is more energy efficient than other more complex parallel architectures (e.g. MIMD)

- only needs to fetch one instruction per data operation
- Regular data-parallel workloads, one instruction controls several datapaths, reducing instruction/control overhead

“SIMD is often exposed via vectorizing compilers and instruction set extensions, which let programmers write code in a sequential style while compilers generate SIMD instructions.

Historical highlight: Important historical SIMD system: ILLIAC IV

(1972 deployment, design started in 1966)



SIMD Architecture

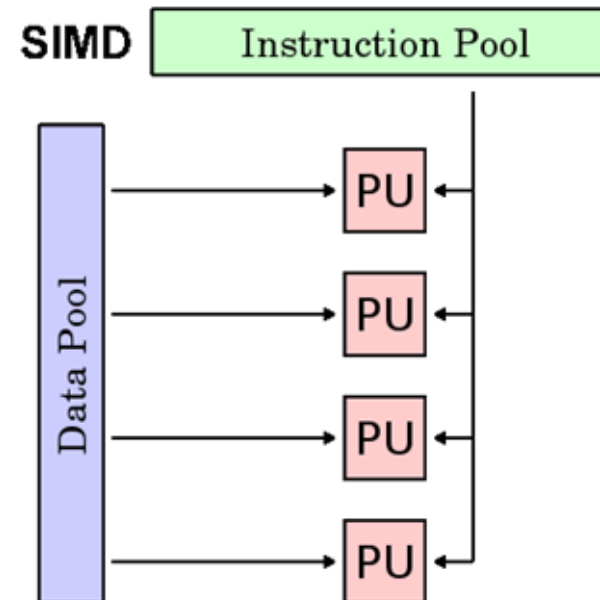
- **Single Instruction:** one instruction is broadcast to all active datapaths. Multiple data: datapaths operate on different elements
- **Multiple Data:** each processing unit can operate on a different data element
- **Synchronous (lockstep) and deterministic execution**

Intuitively

$$z[0..3] = x[0..3] + y[0..3]$$

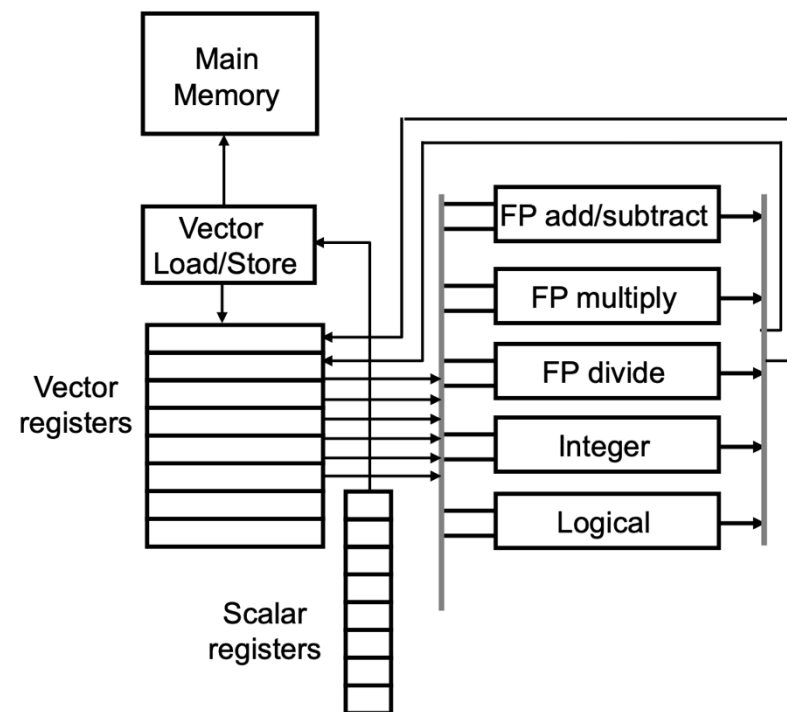
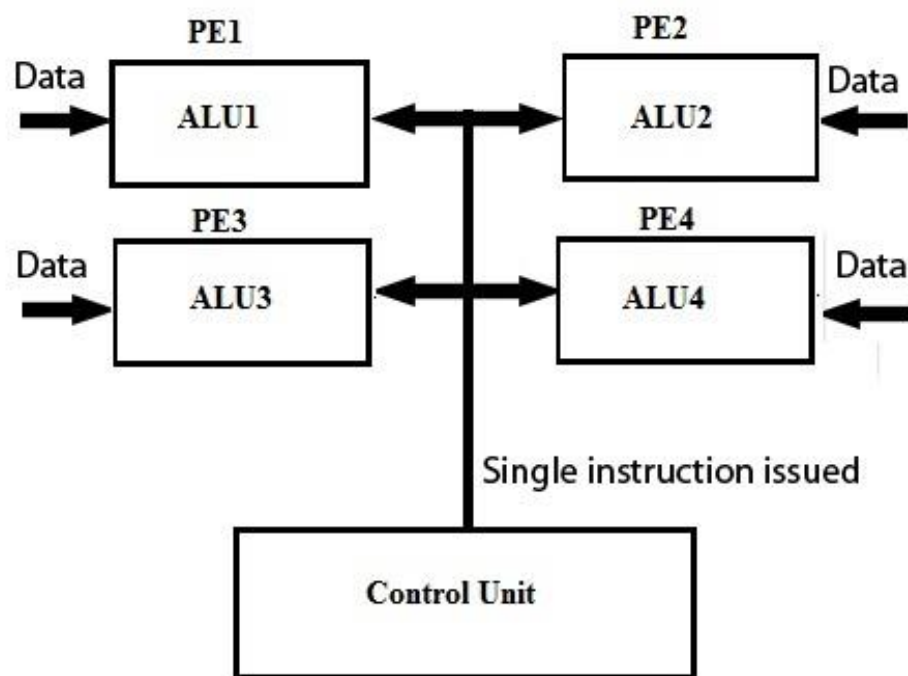
0.5	1.2	-1.0	2.3	0.2	4.5	-3.0	6.1
1.0	0	2.0	2.0	3	4.5	1.0	3.9
=							
-	-	-	-	-	-	-	-

SIMD



X	X ₀
	X ₁
	X ₂
	X ₃
Y	Y ₀
	Y ₁
	Y ₂
	Y ₃

- **Array processors** employ several processing elements to operate in parallel. Classical SIMD / array organization – one control unit, multiple datapaths
- **Vector processors** perform a single instruction on a larger set of data elements, typically a vector register

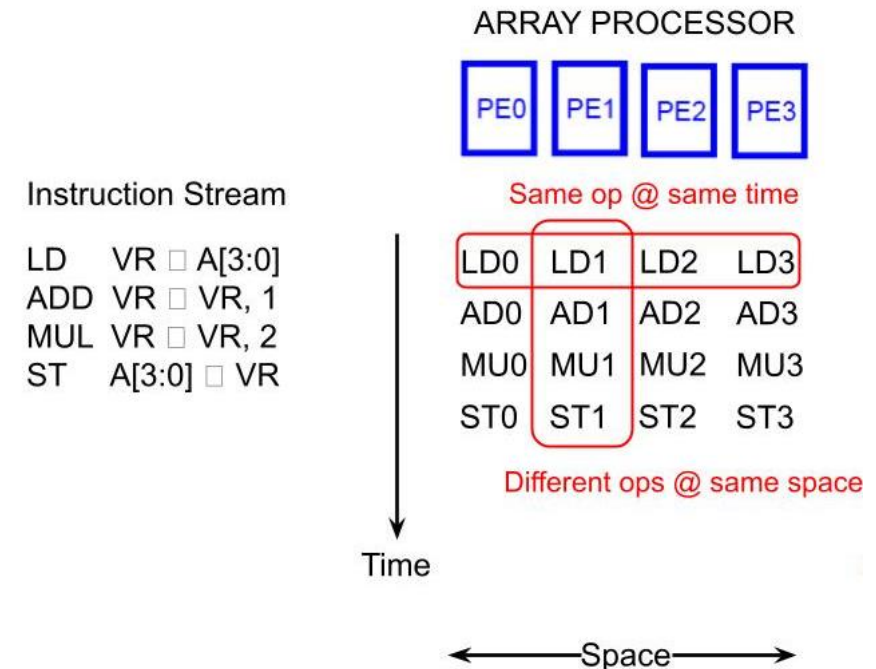
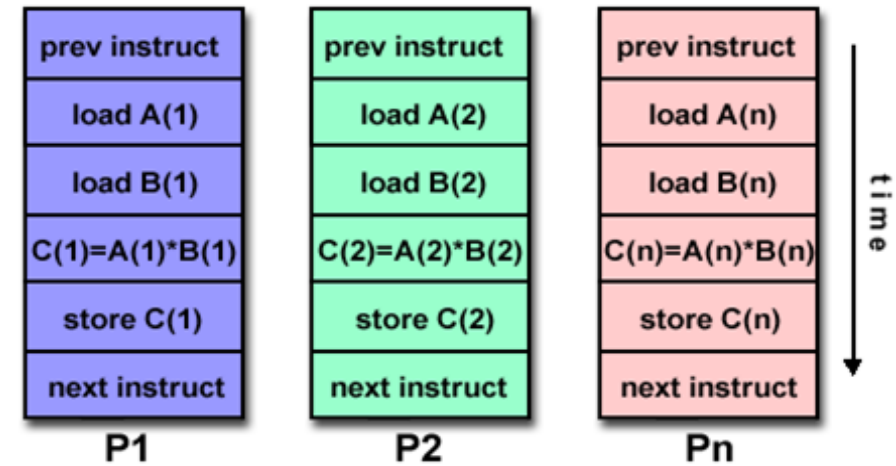


SIMD: Array Architectures

- Each PE in an array processor consists of an **ALU**, **local memory**, and other necessary components. These PEs operate concurrently on different data elements.

FDE and data path:

- Data are loaded into the PEs from the main memory, and after processing, results are stored back.
- All active datapaths execute the same operation on different elements."

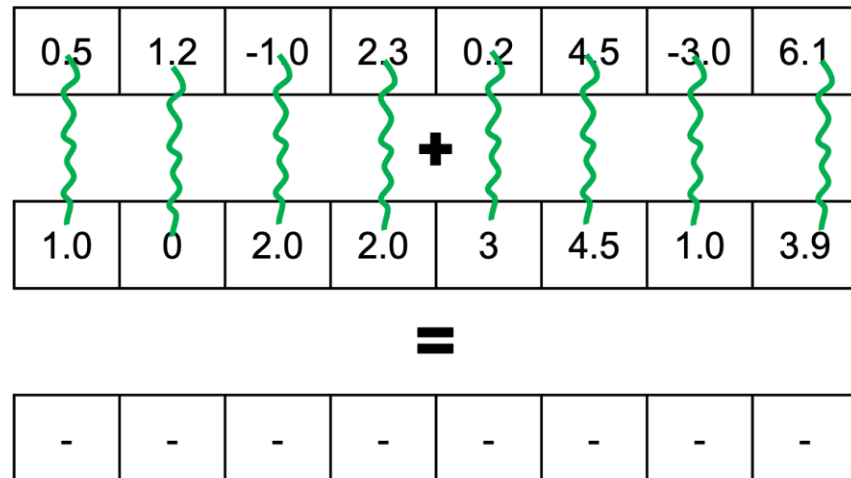
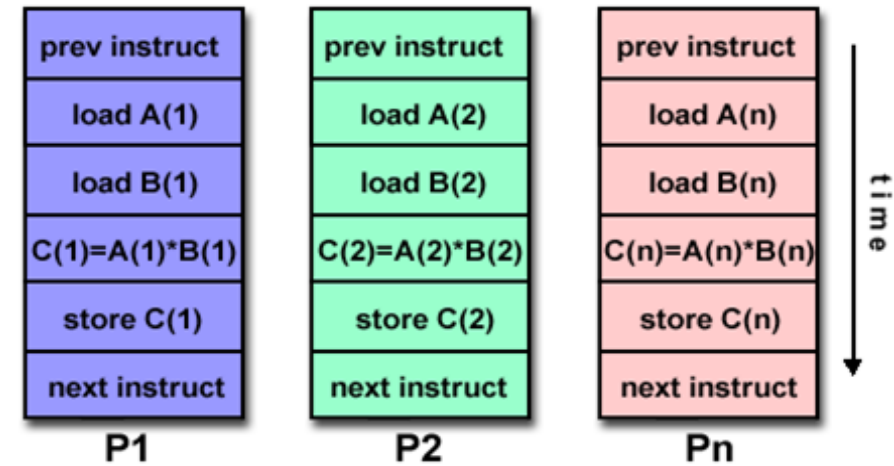


SIMD: Array Architectures

- Each PE in an array processor consists of an **ALU**, **local memory**, and other necessary components. These PEs operate concurrently on different data elements.

FDE and Data Path:

- Data are loaded into the PEs from the main memory, and after processing, results are stored back.
- Each PE typically works on its chunk of data



Example:

Four processing elements with one 32-bit ALU each

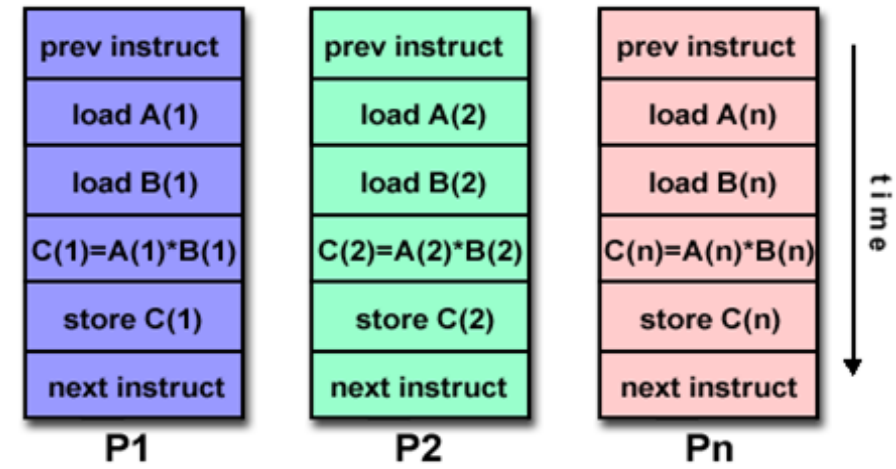
Parallelism degree: 8 independent operations

SIMD width = 4 datapaths → up to 4 element operations concurrently."

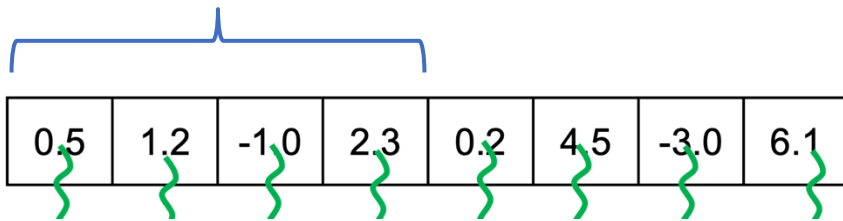
SIMD: Array Architectures

FDE and Data Path

- Data are loaded into the PEs from the main memory
- Each PE works on its chunk of data



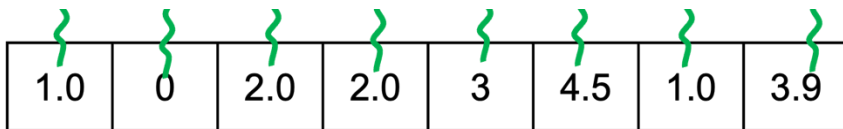
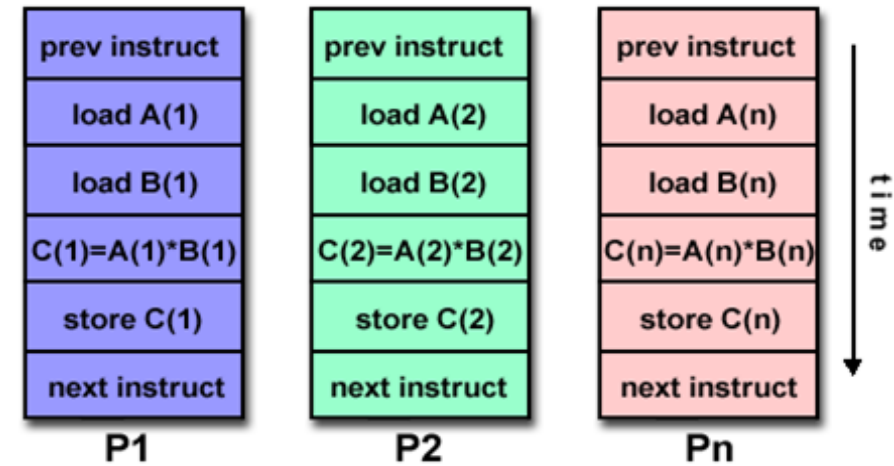
OP1: One **LOAD** instruction is broadcast. Four datapaths load four elements (Operand A)



SIMD: Array Architectures

FDE and Data Path

- Data are loaded into the PEs from the main memory
- Each PE works on its chunk of data. In the example, the operands B are loaded.

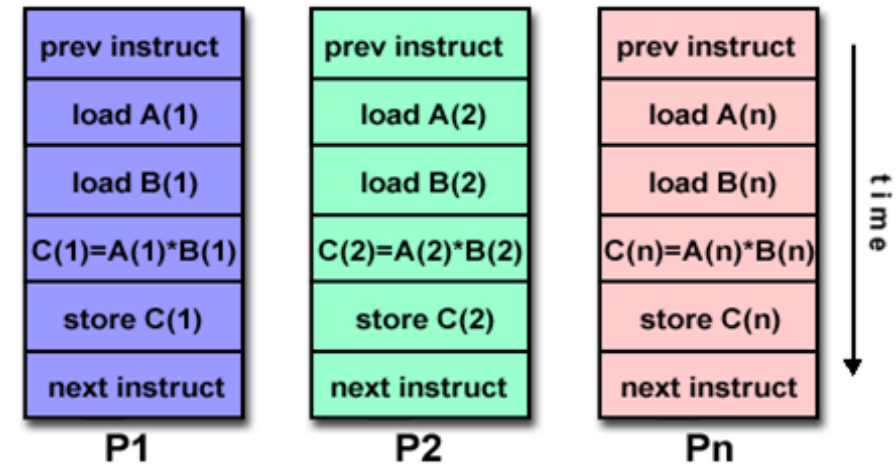


OP2: One **LOAD** instruction is broadcast → four B elements are loaded. (operand B)

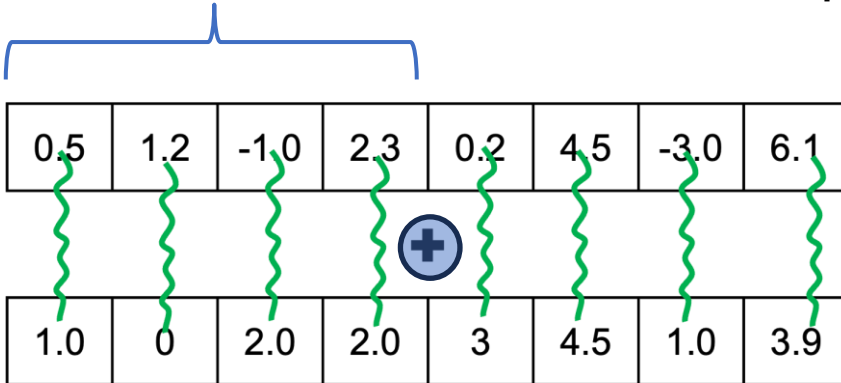
SIMD: Array Architectures

FDE and Data Path

- Each ALU performs the addition

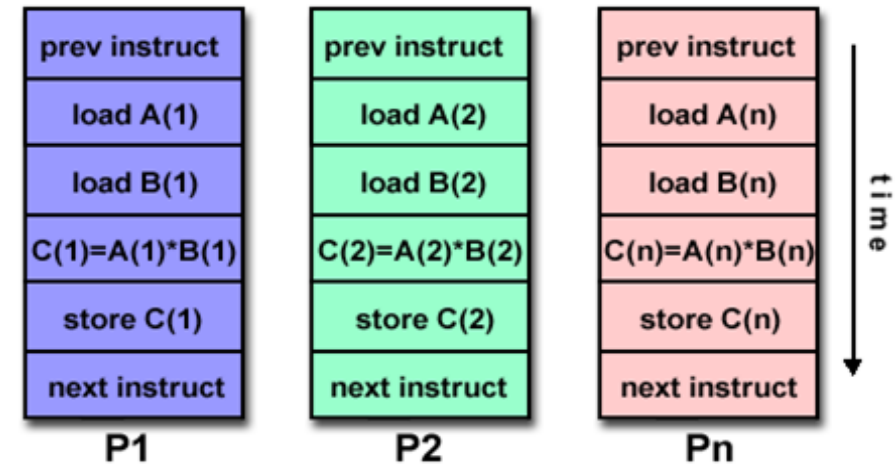
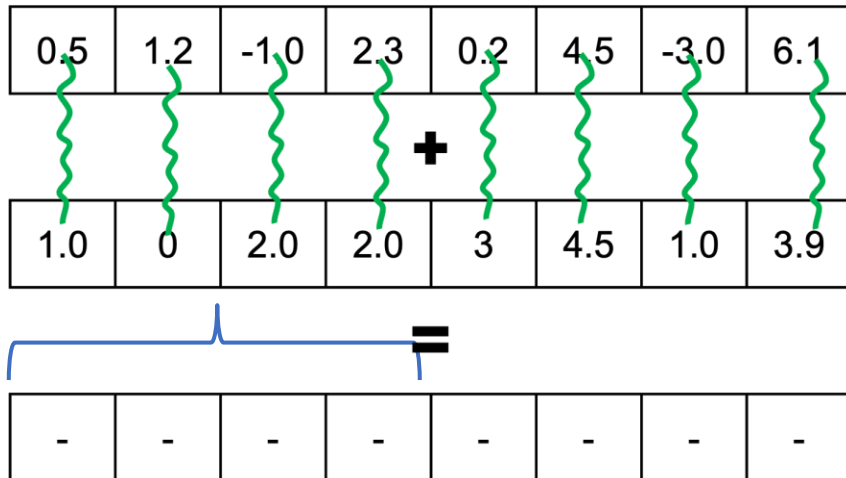


OP3: 4 One ADD instruction → four additions performed concurrently.



FDE and Data Path

- OP4: One STORE instruction is broadcast → four results stored (result)

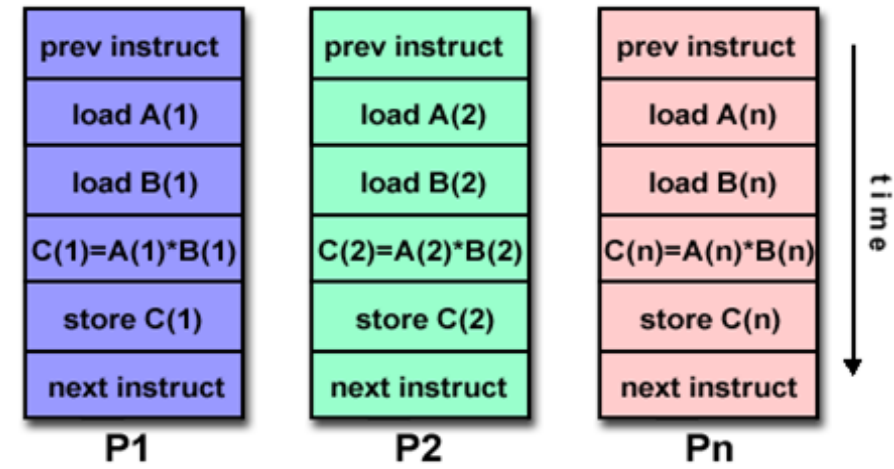


SIMD: Array Architectures

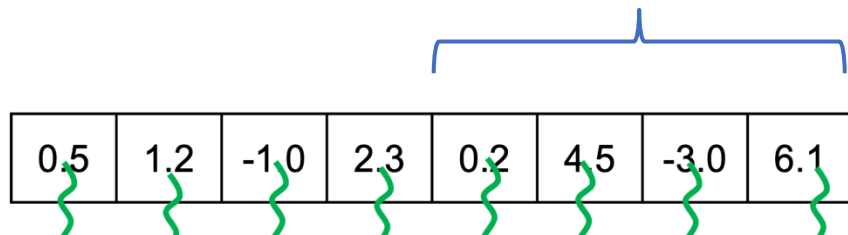
- The memory address is computed by considering the PEs

Example

batch 1 = $A[0, \dots, 3]$
 batch 2 = $A[4, \dots, 7]$
 batch 3 = $A[8, \dots, 11]$
 batch 4 = $A[12, \dots, 15]$



OP5: loads dispatched by the control unit (operand A: 32*4 bit of increment on the base address)



SIMD: Vector Architectures

Vector processors rely vector registers; vectorized/pipelined functional units; vector instructions; vector load/store support

Main characteristics

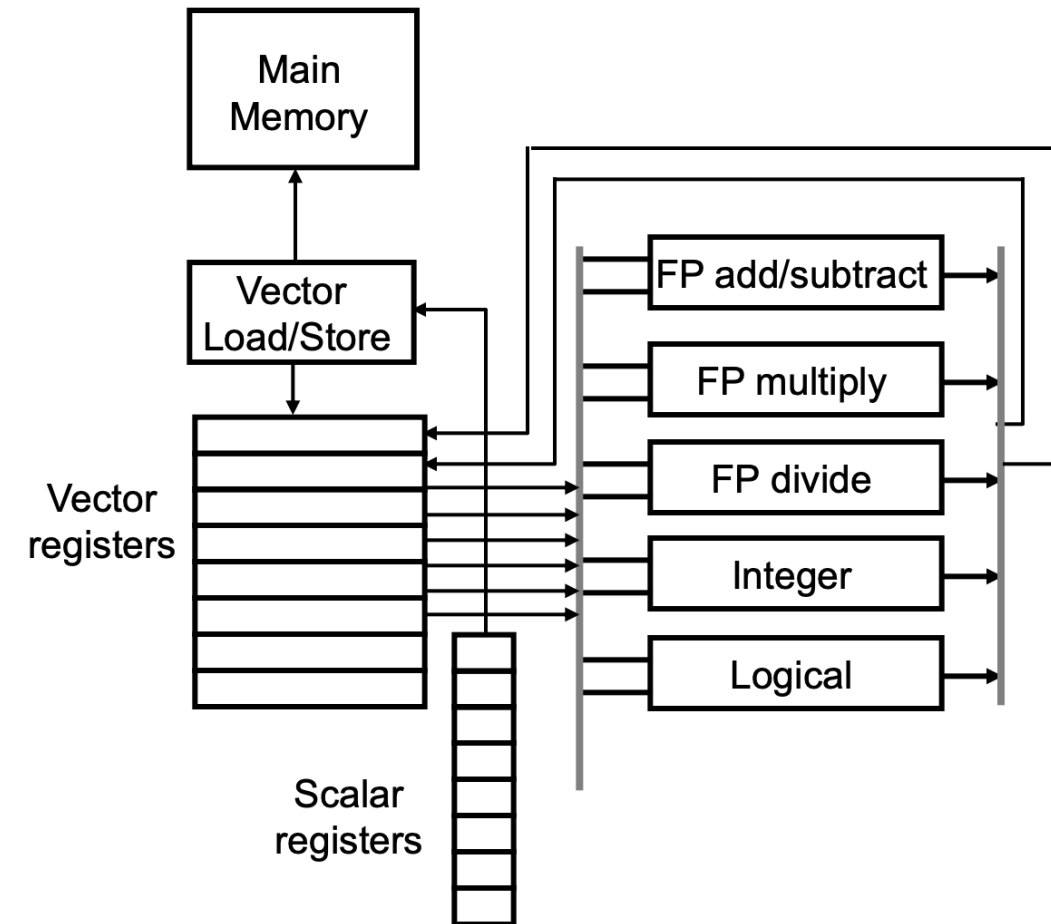
- vector registers (e.g., 64-bit or 128-bit)
- vector functional units
- vector load-store unit
- Scalar registers (32-bit)

Basic idea:

- Read sets of data elements into “vector registers”
- Operate on those registers
- Disperse the results back into memory

Compilers can generate vector instructions and allocate vector registers.

- Used to hide memory latency
- Leverage memory bandwidth



SIMD: Vector Architectures

Example of vector ISA notation

- ADDVV.D: add two vectors
- ADDVS.D: add vector to a scalar
- LV/SV: vector load and vector store from address

# C code	# Scalar Code	# Vector Code
for (i=0; i<64; i++) C[i] = A[i] + B[i];	LI R4, 64 loop: L.D F0, 0(R1) L.D F2, 0(R2) ADD.D F4, F2, F0 S.D F4, 0(R3) DADDIU R1, 8 DADDIU R2, 8 DADDIU R3, 8 DSUBIU R4, 1 BNEZ R4, loop	LI VLR, 64 LV V1, R1 LV V2, R2 ADDV.D V3, V1, V2 SV V3, R3

SIMD: Vector Architectures

Implementations

- **Vector Register/Memory**

Vector can operate faster on small pieces of data: high latency for larger batches
The execution time depends on the length of the operand vectors

- **Vector instructions allow self-looping over larger batches**

Compact: one vector instruction encodes operations on VL elements

Vector Load Operations: when loading vectors contiguous/unit-stride access is generally the simplest way to feed a vector unit efficiently

Advantages:

Great for dense, structured data operations.

Highly optimised for tasks involving repetitive calculations over large data sets.

SIMD: Vector Architectures

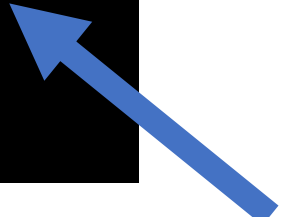
```
void iaxpy(size_t n, int a, const int x[], int y[]) {  
    for (size_t i = 0; i < n; i++)  
        y[i] = a * x[i] + y[i];  
}
```

```
; Initialize loop variables  
mov r1, #0      ; r1 is the loop index  
mov r2, #4      ; r2 is the loop limit (4 in this example)  
  
; Load 4 values from vector register vx into register q1  
movq q1, [vx, r1, lsl #4]  
  
; Load 4 values from vector register vy into register q2  
movq q2, [vy, r1, lsl #4]  
  
; Multiply the values in q1 by va and add the values in q2  
mulps q1, q1, va  
addps q2, q2, q1  
  
; Store the result back to vector register vy  
movq [vy, r1, lsl #4], q2
```

Vector register (128bit / Istr 128bit)

```
; Initialize loop variables  
mov r1, #0      ; r1 is the loop index  
mov r2, #16     ; r2 is the loop limit (16 in this example)  
  
loop:  
    ; Load 4 values from vector register vx into register r3  
    vld1.32 {d3}, [vx]!  
  
    ; Load 4 values from vector register vy into register r4  
    vld1.32 {d4}, [vy]!  
  
    ; Multiply the values in r3 by va and add the values in r4  
    vmul.f32 q1, q1, va  
    vadd.f32 q1, q1, q2  
  
    ; Store the result back to vector register vy  
    vst1.32 {d4}, [vy]!  
  
    ; Increment the loop index  
    add r1, r1, #1  
  
    ; Check if we've reached the loop limit  
    cmp r1, r2  
    bne loop
```

Registers (32bit/ Istr 32)



More compact but high latency for
loading the data

SIMD: Vector Architectures

```
void iaxpy(size_t n, int a, const int x[], int y[]) {  
    for (size_t i = 0; i < n; i++)  
        y[i] = a * x[i] + y[i];  
}
```

```
; Initialize loop variables  
mov r1, #0      ; r1 is the loop index  
mov r2, #4      ; r2 is the loop limit (4 in this example)  
  
; Load 4 values from vector register vx into register q1  
movq q1, [vx, r1, lsl #4]  
  
; Load 4 values from vector register vy into register q2  
movq q2, [vy, r1, lsl #4]  
  
; Multiply the values in q1 by va and add the values in q2  
mulps q1, q1, va  
addps q2, q2, q1  
  
; Store the result back to vector register vy  
movq [vy, r1, lsl #4], q2
```

Vector register (128bit / lstr 128bit)

```
; Initialize loop variables  
mov r1, #0      ; r1 is the loop index  
mov r2, #16     ; r2 is the loop limit (16 in this example)  
  
loop:  
    ; Load 4 values from vector register vx into register r3  
    vld1.32 {d3}, [vx]!  
  
    ; Load 4 values from vector register vy into register r4  
    vld1.32 {d4}, [vy]!  
  
    ; Multiply the values in r3 by va and add the values in r4  
    vmul.f32 q1, q1, va  
    vadd.f32 q1, q1, q2  
  
    ; Store the result back to vector register vy  
    vst1.32 {d4}, [vy]!  
  
    ; Increment the loop index  
    add r1, r1, #1  
  
    ; Check if we've reached the loop limit  
    cmp r1, r2  
    bgt loop
```

Vector instruction set (32bit/ lstr 32)

More complex but you can use pipeline and
prefetching buffer techniques

Issues of SIMD architectures

Vector Architectures Memory addressing implications

Assume row-major storage. For fixed i and j , the inner loop traverses k .

Matrix multiplication

$$C[i][j] = \sum_k A[i][k] \cdot B[k][j]$$

```
for (i = 0; i < N; ++i)
  for (j = 0; j < N; ++j) {
    C[i][j] = 0.0;
    for (k = 0; k < N; ++k)
      C[i][j] += A[i][k] *
B[k][j];
  }
```

Vectorization target: the k loop performs the same multiply-add pattern repeatedly.

What does k access?

$A[i][k]$ → one row of B

$A[i][0]$

$A[i][1]$

$A[i][2]$

$A[i][3]$

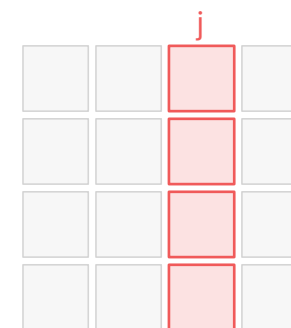
contiguous elements in row-major memory

natural vector load



$B[k][j]$ → one column of B

As k changes, successive values $B[k][j]$ are separated by one row in memory.



$B[0][j]$

$B[1][j]$

$B[2][j]$

$B[3][j]$

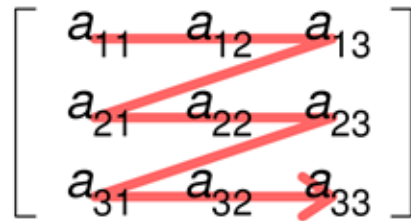
Key idea: vectorizable arithmetic does not automatically imply efficient data movement. Contiguous, appropriately aligned data are usually easier to feed into vector registers.

Vector Architectures Memory addressing implications

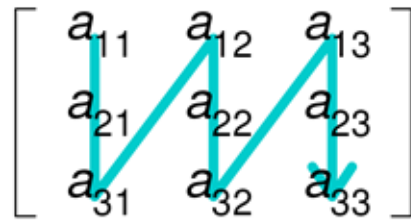


UNITRENTO

Row-major order



Column-major order



C and C++ are row-major

Fortran is column-major

Summary on Vectorial processors

Vector architectures typically load data in chunks (vectors) into registers to process multiple elements simultaneously.

If the memory access patterns are irregular or inefficient, this can lead to performance degradation.

- **Non-contiguous memory access:** when data is scattered across memory, accessing it efficiently becomes difficult. This may lead to **cache misses**, more memory accesses, and longer wait times to fetch the data.
- **Strided memory access:** in certain cases, data is accessed in strides, meaning there is a fixed gap between consecutive memory elements being loaded. Strided access can degrade performance if the memory is not aligned correctly or if the stride length is too large.
- **Memory alignment:** Some vector architectures perform best when data is aligned to certain memory boundaries (e.g., 16- or 32-byte boundaries). Misaligned data can require multiple memory accesses, lowering performance.

Branch Divergence

- Branch divergence causes serialisation
- inactive lanes/ datapaths are masked out, lowering efficiency.

Some PEs take the “if”, others the “else”

Takeaway

Divergence lowers lane utilization because all datapaths cannot execute different instructions simultaneously.

```
if (x[i] > 0)
    y[i] = a[i];

else
    y[i] = b[i];
```

Summary and takeaway

Vector/SIMD key takeaways.

1. regular data parallelism;
2. vector registers/instructions;
3. one instruction operates on several elements;
4. memory must supply elements efficiently.